ASSIGNMENT 2 / LAB 1

How to Create a Stablecoin

Implementation, security analysis and evidence

Yu Liu

16 October 2026

Submission at a glance

github.com/markusliu04/stablecoin-lab-2026

All required Ex0-Ex6 work, the local Ex3 role-compromise demonstration, optional Ex7, and the Sepolia deployment are complete. The full test run records **38 passed, 0 failed, 0 skipped**.

Assessment component	Weight	Included evidence
Contract correctness + test coverage	40%	38 passing tests; full terminal screenshot
Deployment + source verification	20%	3 Sepolia addresses, public receipts and verified-source links
Threat-model analysis	25%	All A-D discussion answers; Ex4 test mapping and final attack scenario
Code style + documentation	15%	Repository, architecture diagram and submission documentation

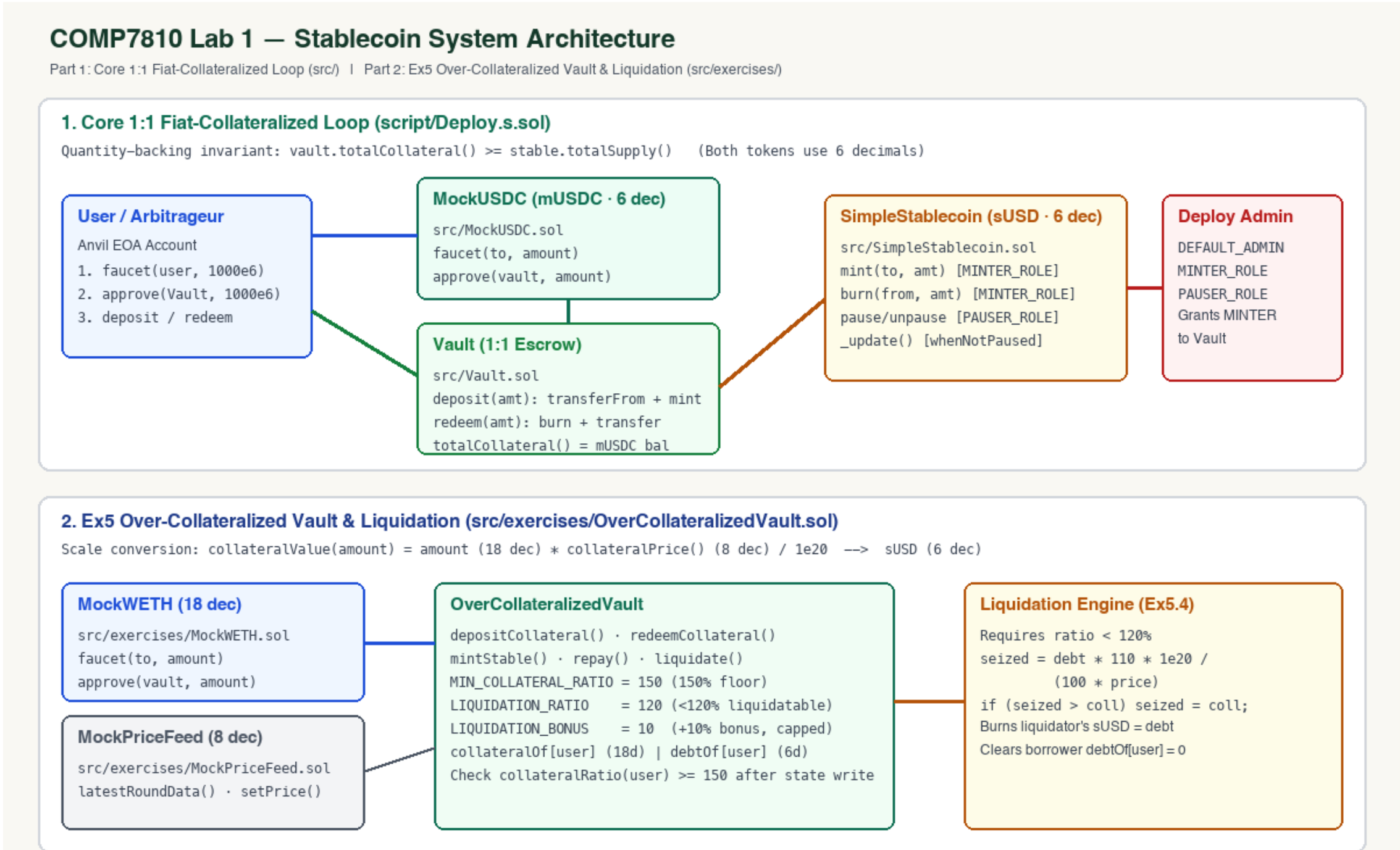
Evidence guide

Architecture (p. 2) / Full test run (p. 3) / Local Ex3 readback (p. 4) / Sepolia deployment (p. 5) / Discussion answers (pp. 6-9) / Test mapping and final scenario (p. 10).

The separate COMP7810_Assignment2_Generative_AI_Appendix.pdf records human and AI contributions. This educational prototype is not audited or production-ready, and token-balance equality does not establish a dollar market peg.

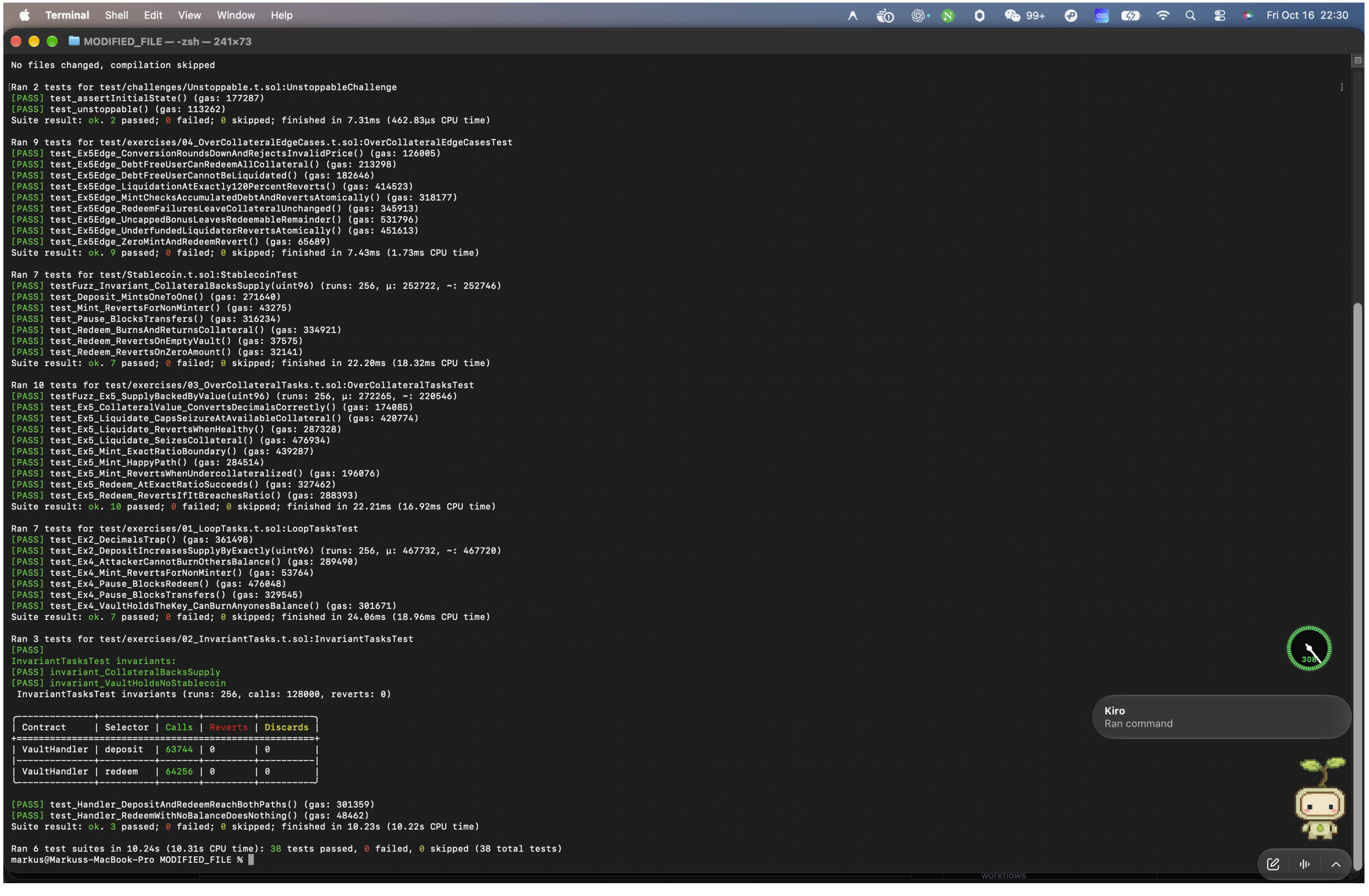
The mint-redeem loop and Ex5 extension

The core deployed system uses 6-decimal mUSDC and sUSD. Ex5 is a separate local over-collateralized system with an 18-decimal collateral token and an 8-decimal price feed.



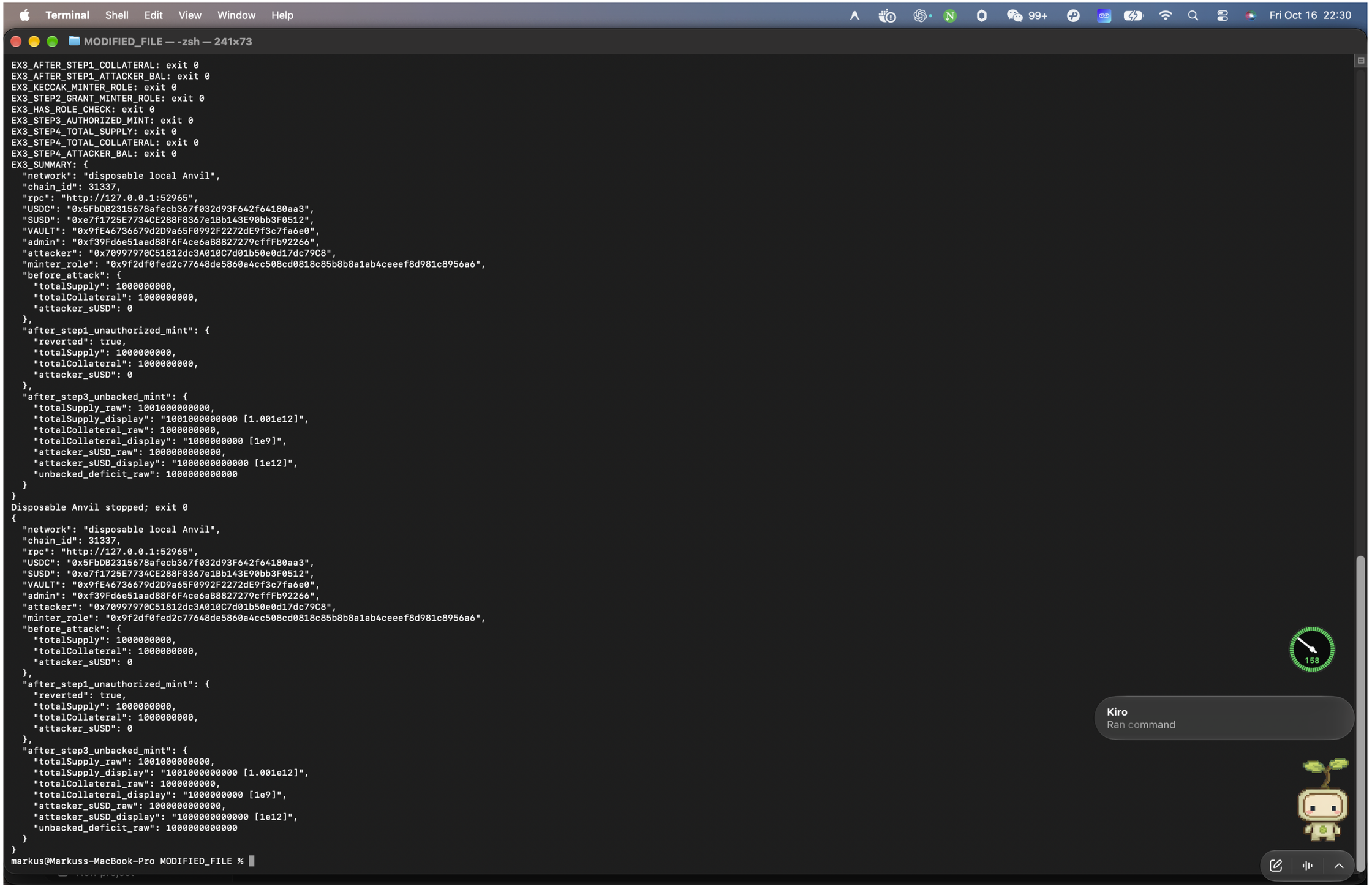
Full test run - 38 passed, 0 failed

Native macOS Terminal capture. Six suites; 38 total tests, 0 failed and 0 skipped. The unfiltered forge test result includes core, exercises, invariants and Ex7.



Ex3 - authorized unbacked mint, unchanged collateral

Disposable local Anvil, chain ID 31337. Unauthorized mint first reverts; after the role grant, totalSupply = 1001000000000 while totalCollateral = 1000000000 raw units. This is not a Sepolia attack.



Public deployment and verification

Network: Sepolia | **Chain ID:** 11155111 | **Block:** 11928229

All three deployment receipts and the role-grant receipt have status **1**. Public readback confirms the Vault references the submitted collateral and stablecoin addresses and holds **MINTER_ROLE**.

Contract	Sepolia contract address	Source status
MockUSDC	`0xdB7bdC4faBE2c136896A061F905227567c5Cc307` Etherscan verified source	Similar Match
SimpleStablecoin	`0x9Acfcc3d676c77E409E81Cf24d2766bEeEe66878` Etherscan verified source	Similar Match
Vault	`0xC940A20D69a12D8806C7d8928a99b41478f266E9` Etherscan verified source	Exact Match

Exact Match versus Similar Match

Vault is an **Exact Match**. MockUSDC and SimpleStablecoin are **Similar Matches**, not Exact Matches. For both Similar Matches, independently retrieved on-chain runtime bytecode matches the local compilation. Source verification alone is not a security audit.

Transaction evidence

Action	Public transaction hash (click to inspect)
MockUSDC deployment	`0x3f41943277c41796377c064cc8508c4caa1ddb3fb1d830ae07c0ced7626d81e6`
SimpleStablecoin deployment	`0xe28ba26fc205ea702e6ca618d12faa68d35b4dea3a68bb92e06d8b3f359a0b23`
Vault deployment	`0x8930288dd342d6bf44621056ee129757a85c4f15cc9e6e0dd4dffc206627b189`
MINTER_ROLE to Vault	`0xa95d12fe61ff189f09f81ffd19d7c2c3aff4097b56207b5e3d5d39777a5225fb`

Deployer / admin

`0x00296F9fb632d058465Cf2f03F496F47237637C2`

Readback recorded 16 October 2026, 23:49 (UTC+08:00). Evidence is a dated observation of the public chain and explorer, not a claim that the source verification labels are interchangeable. No private keys or API credentials are included.

A. Permission design

A1. The vault holds `MINTER_ROLE`, so it can `burn` any user's balance. Explain why that is a risk, then write out how you would change `Vault` and `SimpleStablecoin` to remove it.

The risk is excessive authority, not a missing access-control check. `SimpleStablecoin.burn(from, amount)` checks `MINTER_ROLE`, but never requires the owner's consent or an allowance. Every holder of that role can destroy another account's tokens without returning collateral. The deployer also starts with this role. The current, immutable `Vault` exposes no arbitrary-victim burn endpoint: `redeem` passes `msg.sender` as the account to burn. Therefore, a test impersonating the vault proves the token-level permission, not an externally callable exploit against the current vault implementation. Nevertheless, this permission is an unsafe boundary if a role holder is compromised or different code is later granted the role.

I would separate mint authorization from burn consent. Keep `mint` restricted to the authorized collateral vault, remove the unrestricted role-based `burn(address,uint256)` interface, and provide owner-only `burn(amount)` plus allowance-based `burnFrom(owner, amount)`. The latter must spend the owner's allowance before burning. Change `Vault.redeem` to call `burnFrom(msg.sender, amount)` and then return collateral atomically. Users would approve the stablecoin for redemption in that redesigned system; the current lab does not require that approval. Add tests for missing, insufficient, and exhausted allowance, and prove no minter can burn an unrelated user's balance. These are proposed production changes, not changes made to the educational core here.

A2. In this contract `DEFAULT_ADMIN_ROLE`, `MINTER_ROLE` and `PAUSER_ROLE` all go to the same address. How would you split them in production, and who holds each?

Production should not use one everyday wallet for all three powers. I would place `DEFAULT_ADMIN_ROLE` under a governance multisignature with independent signers and a documented, delayed role-change process. This role is the ultimate authority because it can grant and revoke the other roles; splitting addresses alone does not prevent a malicious admin from granting itself minting power. Admin changes need monitoring, review, and a tested recovery procedure, rather than merely a different label on the same key.

For the 1:1 system, `MINTER_ROLE` should normally belong only to the reviewed vault contract. After deployment and verification of the wiring, revoke the deployer's minting role. Do not give routine treasury operators a general-purpose mint key. Because the current role also permits arbitrary burning, its scope must be redesigned as described in A1 before this becomes a production recommendation.

Give `PAUSER_ROLE` to a separate incident-response multisignature whose signers can act quickly, but cannot change roles or mint. Define when it may pause, who reviews the incident, and how unpausing is approved. Alerts should track role grants, revocations, minting, burning, and pauses. Finally, rehearse compromised-key and lost-signer scenarios. Separation reduces accidental misuse and single-key risk; it does not replace governance or collateral controls.

B. Pausing and redemption

B1. `_update` is the single entry point for every balance change, so `pause()` freezes transfers, minting and redemption together. If you wanted "pause transfers but allow redemption", how would you change it? Give the approach - full code not required.

The current `_update` uses `whenNotPaused`, so it rejects every token balance change: ordinary transfers, minting from the zero address, and burning to the zero address. Redemption fails at `stable.burn`, before collateral is returned. To pause only transfers, remove that blanket modifier and reject an update while paused only when both `from` and `to` are nonzero. Continue calling `super._update` for permitted updates so supply accounting and ERC-20 events remain correct.

Minting policy needs a separate decision. If the desired emergency state is “no transfers or new issuance, but exits remain available,” also gate the external `mint` function with a mint-specific pause check, while permitting authorized redemption burns. Allowing all zero-address updates without a separate mint guard would unintentionally allow new minting during the pause. Burn authorization must still enforce owner consent; exemption from a pause is not permission to burn someone else's balance.

The vault should burn directly and then release collateral. A redesign that first transfers sUSD into the vault would fail during a transfer pause. Tests should prove a transfer and a deposit revert, a valid redemption succeeds, unauthorized burns still revert, and an unsuccessful collateral transfer rolls back the burn. Finally, this token policy cannot force an independently paused or unavailable collateral token to transfer. Redemption availability depends on both contracts and the actual reserve asset.

B2. In 2008, when a money-market fund "broke the buck", redemptions were frozen for days. In 2023 USDC depegged to $0.87 after a reserve bank failed, but redemptions were not shut. Compare the two responses - what does closing the redemption channel, or leaving it open, do to a stablecoin?

The premise needs qualification. The Reserve Primary Fund was a money-market fund, not an on-chain stablecoin. SEC staff described its September 2008 NAV falling to $0.97. The SEC’s September 22 order permitted suspension of redemptions and unpaid redemption proceeds, effective September 17, to protect holders and enable orderly liquidation. That was more than a brief technical pause. SEC account, SEC order.

For USDC, retaining a 1:1 redemption commitment did not mean uninterrupted weekend dollar settlement. Circle’s statement dated March 12 said USDC remained redeemable 1:1 and that the SVB reserve deposit would become available when banks opened. Its March 15 operations update acknowledged disrupted services and redemption backlogs, reporting $3.8 billion redeemed since Monday morning. Thus, “redemptions were not shut” is too categorical if it means cash payouts continued continuously. Circle reserve statement, Circle operations update.

Economically, a credible, executable redemption channel gives arbitrageurs a reason to buy below peg and redeem at par. A closure removes that immediate price anchor, although it may avoid forced asset sales or unequal payouts during liquidation. Leaving a promise open cannot cure insolvency, settlement delays, or restricted customer access. The design lesson is to distinguish solvency, redemption eligibility, operational liquidity, and settlement timing; the lab’s single pause flag hides those distinctions.

C. Depeg analysis

C1. Under what conditions does this coin depeg? Distinguish at least two classes of cause, and say how each one shows up in the invariant `totalCollateral() >= totalSupply()`.

A depeg is a market-price deviation, so this lab cannot observe one merely by reading token balances: it has no exchange or dollar-price oracle. Its quantity invariant is a useful accounting alarm, not a complete price model. The first class of cause is insufficient token backing. For example, an authorized minter issues sUSD without depositing collateral, or a collateral asset unexpectedly loses units. Then `totalCollateral() < totalSupply()`, and the quantity invariant directly detects the shortage. Honest deposit and redeem operations cannot repair an existing deficit simply by reducing both sides equally.

A second class is blocked or costly redemption. A pause freezes the burn required by `Vault.redeem`; unavailable collateral transfers or transaction costs can also prevent an effective exit. The vault can still hold exactly as many collateral units as outstanding sUSD, so `totalCollateral() >= totalSupply()` remains true while the arbitrage mechanism is unavailable. Holders may discount a claim they cannot promptly redeem.

A third distinction is collateral value rather than collateral quantity. If genuine USDC itself traded below one dollar, equal USDC and sUSD token counts would not prove dollar backing. Here `MockUSDC` has an unrestricted faucet and no external economic value, so a local equality test proves only the programmed loop. Operational controls should monitor reserve value and redemption performance separately from on-chain supply, and avoid presenting this educational invariant as proof of a market peg.

C2. Suppose an attacker bribes their way to `MINTER_ROLE`, mints 1,000,000 sUSD out of nothing and redeems it all. Describe the flow of funds, and name the step that could have stopped them.

Assume the attacker has acquired `MINTER_ROLE` and the vault already contains at least 1,000,000 mUSDC from honest depositors. The attacker calls `stable.mint(attacker, 1_000_000e6)` directly. Supply rises by one million sUSD, but the vault receives no collateral, so its backing deficit immediately grows by that amount. The attacker then calls `vault.redeem(1_000_000e6)`. The vault checks its collateral balance, burns the caller's newly issued tokens using its own minter role, and transfers the same number of collateral units to the attacker.

The burn does not undo the economic theft. Supply and collateral both decrease by one million during redemption, so the deficit introduced by the unauthorized issuance remains. Honest users still hold their tokens, but the collateral that would pay them has left the vault. If the vault initially held exactly one million, it ends with zero collateral and one million honest-holder sUSD outstanding. If it held less, that particular full redemption reverts with `InsufficientCollateral`; the attack does not create collateral out of nothing.

The most important preventive step is the role-grant boundary: keep minting restricted to a reviewed collateral-backed path and prevent an insider from assigning general mint authority. Independent admin signers, delayed grants, issuance limits, and alerts add defense in depth. The current access check cannot stop a caller whom governance has already authorized; an invariant read is not an automatically enforced circuit breaker.

D. Toward RWA

D1. Right now the collateral is `MockUSDC` and `totalCollateral()` just reads an on-chain balance - simple and reliable. If the collateral were US Treasuries, could this invariant still be written that way? What new problems appear?

US Treasuries do not become an ERC-20 balance merely because the vault wants to hold them. If the vault holds a tokenized Treasury claim, `balanceOf(vault)` counts claim units, not necessarily immediately redeemable dollars or verified ownership of the underlying securities. A fund-share token can also have a changing net asset value. Therefore the present equality of two six-decimal token quantities is not the right economic backing statement.

Conceptually, I would compare conservatively valued, legally enforceable reserve claims plus cash, less senior liabilities and expenses, against outstanding sUSD obligations in a common unit. An on-chain contract could enforce limits against authenticated reports or a suitable asset token, but it cannot independently inspect a custodian's records. Reconcile reported holdings with custody statements and outstanding claims, investigate double pledging, and reject stale valuation inputs. An attestation is a dated observation, not a perpetual guarantee.

Even short-duration securities introduce sale-price, interest-rate, custody, settlement, and operational risks. A reserve may be solvent at fair value but lack cash for immediate redemptions over a banking weekend. Separate a solvency metric from a liquidity ladder and test stressed liquidation prices and settlement delays. Use conservative mint caps and cash buffers rather than promising instant exits from every asset. These are conceptual design considerations; the appropriate ownership structure and holder rights require jurisdiction-specific professional review, not this lab's code.

D2. If the collateral were a building, how would you put it inside this vault? Which off-chain roles or legal structures would you have to introduce?

A building cannot be transferred into a Solidity contract. The vault could hold a token representing a defined claim on an entity or arrangement that owns the property, but the token alone does not establish title, lien priority, or an enforceable right to occupy or sell it. A possible conceptual structure is a special-purpose property-owning entity with documented investor rights. Calling it an SPV does not automatically make it bankruptcy-remote; the legal effect depends on actual documents and the relevant jurisdiction.

Off-chain participants would include the title-holding entity, a property manager collecting rent and paying expenses, an independent valuer, an auditor reconciling records and token issuance, and an administrator or trustee responsible for enforcing the agreed rights. Insurance, title verification, mortgages, taxes, maintenance, and tenant obligations all affect net realizable value. Conflicts between the operator, valuer, and issuer need explicit controls rather than one trusted signature.

The backing metric should subtract debt and costs and apply conservative valuation haircuts. Appraised value is not cash available for today's redemption; selling or enforcing claims can take months. A building-backed product would likely need cash reserves, a disclosed redemption queue, and defined loss-allocation and default procedures, not this vault's unconditional instant 1:1 exit model. An oracle can publish reports but cannot foreclose or guarantee their truth. This is an educational architecture sketch, not legal advice or a claim that the lab has implemented an RWA system.

E. Tests and final attack scenario

Turn the red tests green in `test/exercises/01_LoopTasks.t.sol` to cover the scenarios below, and write your test function names here:

Scenario	Your test function name
Minting by a non-minter reverts	`test_Ex4_Mint_RevertsForNonMinter`
Transfers revert while paused	`test_Ex4_Pause_BlocksTransfers`
Redemption reverts while paused	`test_Ex4_Pause_BlocksRedeem`
An attacker cannot burn someone else's balance	`test_Ex4_AttackerCannotBurnOthersBalance`
...but the vault holding `MINTER_ROLE` can	`test_Ex4_VaultHoldsTheKey_CanBurnAnyonesBalance`

That last pair is meant to be read together: the guard is written correctly, but the key was handed to the vault. Keep it in mind when you answer A1.

Now write one more scenario you consider **most likely to be attacked**, and say why you picked it:

Compromised `MINTER_ROLE` grant and unbacked issuance (demonstrated on local Anvil in Ex3): a compromised admin key or insider role grant lets an external account receive `MINTER_ROLE` and call `stable.mint(attacker, 1_000_000e6)` directly without depositing collateral into `Vault`. I picked this scenario because the token's `onlyRole(MINTER_ROLE)` guard is written as intended, yet a governance/key-management failure bypasses `Vault.deposit` without changing a single line of Solidity code. In our local Anvil Ex3 run, `totalSupply()` jumped from `1,000,000,000` (`1,000` sUSD) to `1,001,000,000,000` (`1,001,000` sUSD) while `vault.totalCollateral()` stayed at `1,000,000,000` (`1,000` mUSDC), breaking the backing invariant and allowing the unbacked holder to drain honest depositors' collateral via `vault.redeem`.

Source scope: all questions and answers from STUDENT-QUESTIONS.md are reproduced in this report. The RWA discussion is conceptual, not legal or investment advice. Primary-source links in B2 remain clickable.